```
    if existing != nil {
        return fmt.Errorf("UAV %s already registered",
uavID)}
    uav := UAVIdentity{uavID, certHash, swarmID, "ACTIVE",
timestamp}
    data, _ := json.Marshal(uav)
    return ctx.GetStub().PutState(uavID, data)}
// RevokeUAV is called automatically when a DoS/U2R attack is
confirmed
func (r *UAVRegistry) RevokeUAV(ctx contractapi.
TransactionContextInterface,
    uavID string) error {
    data, _: = ctx.GetStub().GetState(uavID)
    var uav UAVIdentity
    json.Unmarshal(data, &uav)
    uav.Status = "REVOKED"
    updated, _:= json.Marshal(uav)
    return ctx.GetStub().PutState(uavID, updated)
}
```

After *RegisterUAV()* is called, a JSON output will be as follows:

```
Key: "UAV-001"
Value: {
  "uav_id": "UAV-001",
  "cert_hash": "a3f9d12bc...e7a",
  "swarm_id": "SWARM-ALPHA",
  "status": "ACTIVE",
  "registered_at": "2025-04-13T10:00:00Z"
}
```

If *RegisterUAV()* is called again with the same UAV ID, an error will be thrown as follows:

```
Error: UAV UAV-001 already registered
```

After *RevokeUAV()* is called on UAV, a JSON output will be as follows:

```
Key: "UAV-001"
Value: {
  "uav_id": "UAV-001",
  "cert_hash": "a3f9d12bc...e7a",
  "swarm_id": "SWARM-ALPHA",
  "status": "REVOKED",
  "registered_at": "2025-04-13T10:00:00Z"
}
```

Secure event logging with tamper-proof audit

Every bit of information transferred via UAVs is encrypted using a hybrid scheme of AES-256 and ECC P-256. While the AES-256-GCM handles the payload faster and authenticates the encryption, the ECC P-256 (which is a lightweight algorithm ideal for constrained UAV hardware compared to RSA) handles the key exchange more securely. This combination gave us an encryption overhead of just 2.3ms per message in simulation.

The Python code to attain this is as follows:

```
from cryptography.hazmat.primitives.asymmetric.ec import (
    ECDH, generate_private_key, SECP256R1 )
from cryptography.hazmat.primitives.ciphers.aead import
AESGCM
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography.hazmat.primitives import hashes
import os
def encrypt_telemetry(payload: bytes, recipient_pub_key) ->
dict:
    # Ephemeral ECC key pair for this session only
    ephemeral = generate_private_key(SECP256R1())
    shared_secret = ephemeral.exchange(ECDH(), recipient_pub_
key)
    # Derive AES-256 key via HKDF-SHA256
    aes_key = HKDF(hashes.SHA256(), 32, None, b"uav-swarm").
derive(shared_secret)
    # AES-256-GCM: encrypts + authenticates in one step
    nonce = os.urandom(12)
    ciphertext = AESGCM(aes_key).encrypt(nonce, payload,
None)
    return {"nonce": nonce.hex(), "ciphertext": ciphertext.
hex()
}
```

The output is:

```
=== UAV Telemetry Encryption Output ===
Original Payload : UAV-001 | LAT:13.0827 | LON:80.2707 | ALT:120m | STATUS:ACTIVE
Nonce (12 bytes) : bea04103bc6b5704cb33aa26
Ciphertext       : 3cc194a6bef0048b7291562356dc375814ace398f8323ba
                   ceec81c9474f31fc178c3d67401bf1114cc9b95572bd60
                   9439a68a58d9bb1928ce9929e8f5376593c8b03aeb3f53
                   96ba53ea106658dfe
Nonce Length     : 12 bytes
Ciphertext Length: 78 bytes
```

Results and performance evaluation

The results were obtained by deploying the Hyperledger Fabric testnet on Docker containers using MATLAB simulations with 4 peer nodes, 1 orderer and 1 CA.

Figure 3 depicts that the PBFT provides the optimal balance of 45.2ms, which is faster than all decentralised blockchain alternatives.

Consensus method	Authentication latency
Centralised authentication	18.4ms
PBFT (Proposed)	45.2ms
Raft (Consensus)	62.1ms
PoS (Ethereum)	720.0ms
PoW (Bitcoin)	8,400ms

Table 2: Consensus method and its latency